

# Answerspot — Dockerfiles & Helm Chart

Closes the deploy gap: the images the CI/CD workflows build and the chart they deploy. Paths shown per file.

```
infra/
├─ docker/
│   ├── web.Dockerfile
│   ├── api.Dockerfile
│   └─ worker.Dockerfile
└─ helm/
    ├── Chart.yaml
    ├── values.yaml
    ├── values.staging.yaml
    ├── values.prod.yaml
    └─ templates/
        ├── _helpers.tpl
        ├── configmap.yaml
        ├── web-deployment.yaml
        ├── api-deployment.yaml · api-service.yaml
        ├── worker-runner-deployment.yaml
        ├── worker-parser-deployment.yaml
        ├── worker-recs-deployment.yaml
        ├── scheduler-deployment.yaml
        ├── prisma-migrate-cronjob.yaml
        ├── ingress.yaml
        └─ hpa.yaml
```

## infra/docker/web.Dockerfile

dockerfile

```
# Next.js (standalone output). Set `output: 'standalone'` in next.config.js.
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY apps/web/package.json apps/web/
RUN npm ci

FROM node:20-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build --workspace=apps/web

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -g 1001 nodejs && adduser -u 1001 -G nodejs -S nextjs
COPY --from=build --chown=nextjs:nodejs /app/apps/web/.next/standalone ./
COPY --from=build --chown=nextjs:nodejs /app/apps/web/.next/static ./apps/web/.next/s
COPY --from=build --chown=nextjs:nodejs /app/apps/web/public ./apps/web/public
USER nextjs
EXPOSE 3000
CMD ["node", "apps/web/server.js"]
```

infra/docker/api.Dockerfile

dockerfile

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY apps/api/package.json apps/api/
RUN npm ci

FROM node:20-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate --schema apps/api/prisma/schema.prisma
RUN npm run build --workspace=apps/api

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -g 1001 nodejs && adduser -u 1001 -G nodejs -S nestjs
# Runtime deps only + generated Prisma client + built output.
COPY --from=build --chown=nestjs:nodejs /app/node_modules ./node_modules
COPY --from=build --chown=nestjs:nodejs /app/apps/api/dist ./apps/api/dist
COPY --from=build --chown=nestjs:nodejs /app/apps/api/prisma ./apps/api/prisma
USER nestjs
EXPOSE 4000
# Health endpoint used by K8s probes + CI smoke tests.
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -qO- http://localhost:4000/api/v1/health || exit 1
CMD ["node", "apps/api/dist/main.js"]
```

#### infra/docker/worker.Dockerfile

dockerfile

```
# Shared image for all workers + scheduler. Command overridden per deployment.
FROM python:3.12-slim AS base
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY apps/workers/pyproject.toml ./
RUN pip install --upgrade pip && pip install -e .

COPY apps/workers/ ./

RUN useradd -m -u 1001 celery
USER celery

# Default: runner queue. Overridden in each deployment (parser/recs/beat).
CMD ["celery", "-A", "answerspot_workers", "worker", "-Q", "runner", "--loglevel=info"]
```

---

#### infra/helm/Chart.yaml

```
yaml

apiVersion: v2
name: answerspot
description: AI answer-monitoring platform for local service businesses
type: application
version: 0.1.0
appVersion: "0.1.0"
```

---

#### infra/helm/values.yaml

```
yaml
```

```
# Defaults. Overridden by values.staging.yaml / values.prod.yaml.

image:
  registry: ghcr.io
  repository: your-org/answerspot # overridden in CI via --set is not needed; repo
  tag: latest
  pullPolicy: IfNotPresent

imagePullSecrets: []

# External datastores are managed per ADR 0002 – provide connection via secret.
existingSecret: answerspot-secrets # must contain DATABASE_URL, REDIS_URL, etc.

web:
  replicas: 2
  resources:
    requests: { cpu: 100m, memory: 256Mi }
    limits: { cpu: 500m, memory: 512Mi }

api:
  replicas: 2
  port: 4000
  resources:
    requests: { cpu: 200m, memory: 512Mi }
    limits: { cpu: "1", memory: 1Gi }
  hpa:
    enabled: true
    minReplicas: 2
    maxReplicas: 6
    targetCPUUtilizationPercentage: 70

workers:
  runner:
    replicas: 2
    concurrency: 4
    resources:
      requests: { cpu: 250m, memory: 512Mi }
      limits: { cpu: "1", memory: 1Gi }
    hpa:
      enabled: true
      minReplicas: 2
      maxReplicas: 8
      targetCPUUtilizationPercentage: 70
```

```
parser:
  replicas: 2
  concurrency: 4
  resources:
    requests: { cpu: 250m, memory: 512Mi }
    limits:   { cpu: "1",  memory: 1Gi }
recs:
  replicas: 1
  concurrency: 2
  resources:
    requests: { cpu: 100m, memory: 256Mi }
    limits:   { cpu: 500m, memory: 512Mi }

scheduler:
  replicas: 1 # exactly one beat instance
  resources:
    requests: { cpu: 50m, memory: 128Mi }
    limits:   { cpu: 200m, memory: 256Mi }

ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
  hosts:
    web: app.answerspot.example
    api: api.answerspot.example
  tls: true

config:
  scanSampleCount: "3"
  logLevel: info
```

infra/helm/values.staging.yaml

yaml

```
image:
  tag: staging

web:
  replicas: 1
api:
  replicas: 1
  hpa:
    enabled: false
workers:
  runner: { replicas: 1 }
  parser: { replicas: 1 }
  recs:   { replicas: 1 }

ingress:
  hosts:
    web: staging-app.answerspot.example
    api: staging-api.answerspot.example

config:
  logLevel: debug
```

---

infra/helm/values.prod.yaml

yaml

```
image:
  tag: prod

web:
  replicas: 3
api:
  replicas: 3
  hpa:
    enabled: true
    minReplicas: 3
    maxReplicas: 10
workers:
  runner:
    replicas: 3
    hpa: { enabled: true, minReplicas: 3, maxReplicas: 12, targetCPUUtilizationPerce
  parser: { replicas: 3 }
  recs: { replicas: 2 }

ingress:
  hosts:
    web: app.answerspot.example
    api: api.answerspot.example
```

infra/helm/templates/\_helpers.tpl

yaml



```
{{- define "answerspot.image" -}}
{{ .Values.image.registry }}/{{ .Values.image.repository }}-{{ .service }}:{{ .Values
{{- end -}}

{{- define "answerspot.labels" -}}
app.kubernetes.io/name: answerspot
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end -}}

{{- define "answerspot.envFrom" -}}
envFrom:
  - secretRef:
      name: {{ .Values.existingSecret }}
  - configMapRef:
      name: {{ .Release.Name }}-config
{{- end -}}
```

#### infra/helm/templates/configmap.yaml

yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-config
  labels: {{- include "answerspot.labels" . | nindent 4 }}
data:
  NODE_ENV: production
  LOG_LEVEL: {{ .Values.config.logLevel | quote }}
  SCAN_SAMPLE_COUNT: {{ .Values.config.scanSampleCount | quote }}
```

#### infra/helm/templates/api-deployment.yaml

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-api
  labels: {{- include "answerspot.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.api.replicas }}
  selector:
    matchLabels: { app.kubernetes.io/component: api }
  template:
    metadata:
      labels:
        app.kubernetes.io/component: api
        {{- include "answerspot.labels" . | nindent 8 }}
    spec:
      {{- with .Values.imagePullSecrets }}
      imagePullSecrets: {{- toYaml . | nindent 8 }}
      {{- end }}
      containers:
        - name: api
          image: {{ include "answerspot.image" (dict "Values" .Values "service" "api"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: {{ .Values.api.port }}
          {{- include "answerspot.envFrom" . | nindent 10 }}
          readinessProbe:
            httpGet: { path: /api/v1/health, port: {{ .Values.api.port }} }
            initialDelaySeconds: 10
            periodSeconds: 10
          livenessProbe:
            httpGet: { path: /api/v1/health, port: {{ .Values.api.port }} }
            initialDelaySeconds: 20
            periodSeconds: 15
          resources: {{- toYaml .Values.api.resources | nindent 12 }}
```

infra/helm/templates/api-service.yaml

yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-api
  labels: {{- include "answerspot.labels" . | nindent 4 }}
spec:
  selector: { app.kubernetes.io/component: api }
  ports:
    - port: 80
      targetPort: {{ .Values.api.port }}
```

---

infra/helm/templates/web-deployment.yaml

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-web
  labels: {{- include "answerspot.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.web.replicas }}
  selector:
    matchLabels: { app.kubernetes.io/component: web }
  template:
    metadata:
      labels:
        app.kubernetes.io/component: web
        {{- include "answerspot.labels" . | nindent 8 }}
    spec:
      containers:
        - name: web
          image: {{ include "answerspot.image" (dict "Values" .Values "service" "web"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 3000
            {{- include "answerspot.envFrom" . | nindent 10 }}
          resources: {{- toYaml .Values.web.resources | nindent 12 }}
      ---
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-web
  labels: {{- include "answerspot.labels" . | nindent 4 }}
spec:
  selector: { app.kubernetes.io/component: web }
  ports:
    - port: 80
      targetPort: 3000
```

infra/helm/templates/worker-runner-deployment.yaml

yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-worker-runner
  labels: {{- include "answerspot.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.workers.runner.replicas }}
  selector:
    matchLabels: { app.kubernetes.io/component: worker-runner }
  template:
    metadata:
      labels:
        app.kubernetes.io/component: worker-runner
        {{- include "answerspot.labels" . | nindent 8 }}
    spec:
      containers:
        - name: worker-runner
          image: {{ include "answerspot.image" (dict "Values" .Values "service" "work
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          command: ["celery", "-A", "answerspot_workers", "worker",
                    "-Q", "runner",
                    "--loglevel=info",
                    "--concurrency={{ .Values.workers.runner.concurrency }}"]
          {{- include "answerspot.envFrom" . | nindent 10 }}
          resources: {{- toYaml .Values.workers.runner.resources | nindent 12 }}

```

`worker-parser-deployment.yaml` and `worker-recs-deployment.yaml` are identical to the above with `runner` → `parser` / `recs` in the name, component label, `-Q` queue, and `.Values.workers.*` path.

`infra/helm/templates/scheduler-deployment.yaml`

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-scheduler
  labels: {{- include "answerspot.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.scheduler.replicas }}    # keep at 1 (single beat)
  strategy: { type: Recreate }                  # never two beats at once
  selector:
    matchLabels: { app.kubernetes.io/component: scheduler }
  template:
    metadata:
      labels:
        app.kubernetes.io/component: scheduler
        {{- include "answerspot.labels" . | nindent 8 }}
    spec:
      containers:
        - name: scheduler
          image: {{ include "answerspot.image" (dict "Values" .Values "service" "work" "context" "prisma-migrate-cronjob") }}
          command: ["celery", "-A", "answerspot_workers", "beat", "--loglevel=info"]
          {{- include "answerspot.envFrom" . | nindent 10 }}
          resources: {{- toYaml .Values.scheduler.resources | nindent 12 }}
```

infra/helm/templates/prisma-migrate-cronjob.yaml

yaml

```
# Suspended CronJob used as a template. CI creates a Job from it per deploy
# (see deploy.yml / release.yml: kubectl create job --from=cronjob/prisma-migrate).
apiVersion: batch/v1
kind: CronJob
metadata:
  name: prisma-migrate
  labels: {{- include "answerspot.labels" . | nindent 4 }}
spec:
  schedule: "0 0 31 2 *" # never runs on its own (Feb 31)
  suspend: true
  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: migrate
              image: {{ include "answerspot.image" (dict "Values" .Values "service" "prisma-migrate") }}
              command: ["npm", "run", "prisma:migrate", "deploy",
                "--schema", "apps/api/prisma/schema.prisma"]
              {{- include "answerspot.envFrom" . | nindent 14 }}
```

infra/helm/templates/ingress.yaml

yaml

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ .Release.Name }}-ingress
  labels: {{- include "answerspot.labels" . | nindent 4 }}
  annotations: {{- toYaml .Values.ingress.annotations | nindent 4 }}
spec:
  ingressClassName: {{ .Values.ingress.className }}
  {{- if .Values.ingress.tls }}
  tls:
    - hosts:
        - {{ .Values.ingress.hosts.web }}
        - {{ .Values.ingress.hosts.api }}
      secretName: {{ .Release.Name }}-tls
  {{- end }}
  rules:
    - host: {{ .Values.ingress.hosts.web }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: {{ .Release.Name }}-web, port: { number: 80 } }
    - host: {{ .Values.ingress.hosts.api }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: {{ .Release.Name }}-api, port: { number: 80 } }
  {{- end }}
```

infra/helm/templates/hpa.yaml

yaml



```
{{- if .Values.api.hpa.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ .Release.Name }}-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Release.Name }}-api
  minReplicas: {{ .Values.api.hpa.minReplicas }}
  maxReplicas: {{ .Values.api.hpa.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.api.hpa.targetCPUUtilizationPercentage }}
{{- end }}
{{- if .Values.workers.runner.hpa.enabled }}
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ .Release.Name }}-worker-runner-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Release.Name }}-worker-runner
  minReplicas: {{ .Values.workers.runner.hpa.minReplicas }}
  maxReplicas: {{ .Values.workers.runner.hpa.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.workers.runner.hpa.targetCPUUtilizationPercentage }}
{{- end }}
```

## Wiring notes

- **CI already references these.** `deploy.yml` / `release.yml` build `infra/docker/{web,api,worker}.Dockerfile` and run `helm upgrade ... infra/helm` with the matching values file. This closes that dependency.
- **The migrate CronJob is intentionally suspended** — CI materializes a Job from it per deploy, so migrations run once per release, not on a schedule.
- `existingSecret` must hold `DATABASE_URL`, `REDIS_URL`, `CELERY_BROKER_URL`, `JWT_SECRET`, `STRIPE_*`, and the AI platform keys. Create via Sealed Secrets / Vault per ADR 0002 — never in values files.
- **Scheduler is pinned to 1 replica with `Recreate`** so only one Celery beat ever runs.
- **Image naming** assumes `repository-service:tag` (e.g. `answerspot-api:prod`). Align this with the tag format your registry push in CI produces, or adjust the `answerspot.image` helper.